Graph Topologies and Incentive Stability in Multi-Agent Code Generation

NANA YE*, University of Waterloo, Canada
CHARLOTTE MA*, University of Waterloo, Canada
JASON LIU*, University of Waterloo, Canada

With the rapid advancement in LLMs (Large Language Models), the Multi-agent LLM systems showed strong gains in code generation by decomposing software development into specialized roles and introducing structured feedback. However, existing systems are typically evaluated as holistic pipelines in which many factors change simultaneously: agent roles, prompts, tools, and stopping rules. As a result, it remains unclear how much of the observed performance gain arises specifically from the interaction graph structure. This lack of controlled analysis limits our understanding of how different topologies shape agents' incentives to spend tokens on verification under constrained budgets. This paper aims to evaluate how these graph structures affect the tradeoff between accuracy and compute cost. Specifically, we investigate whether complex graph structures discourage high-effort verification from agents due to varying incentives.

CCS Concepts: • **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software maintenance tools.*

Additional Key Words and Phrases: Multi-Agent Systems, Code Generation, Large Language Models, Graph Topology, Incentive Stability, Game Theory

ACM Reference Format:
Nana Ye, Charlotte Ma, and Jason Liu. 2018. Graph Topologies and Incentive Stability in Multi-Agent Code Generation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Multi-agent large language model systems have recently emerged as a promising paradigm for code generation, motivated by the fact that software development is inherently multi-stage and often benefits from explicit decomposition. Rather than asking a single model invocation to directly produce a correct program, these systems distribute planning, implementation, review, testing, and debugging across specialized roles, creating opportunities for iterative error detection and repair. This shift is grounded in earlier evidence that large language models can perform nontrivial program synthesis while still falling short on functional correctness. In particular, Austin et al. (2021) introduced the Mostly Basic Programming Problems (MBPP) benchmark and showed that even very large models leave substantial room for improvement under test-based evaluation, while also benefiting from dialog and feedback. These observations helped motivate the move from one-shot generation toward agentic and multi-stage coding systems.

*All authors contributed equally to this research.

Authors' Contact Information: Nana Ye, University of Waterloo, Waterloo, Canada, email@domain.com; Charlotte Ma, University of Waterloo, Waterloo, Canada, email@domain.com; Jason Liu, University of Waterloo, Waterloo, Canada, mx5liu@uwaterloo.ca.

Manuscript submitted to ACM

In this paper, we study multi-agent code generation through the lens of graph topology and incentive stability. We evaluate representative interaction graphs on MBPP using a fixed base model and controlled role prompts, with the goal of isolating how structural differences affect both functional correctness and computation cost. Rather than proposing yet another end-to-end coding framework, we treat topology itself as the primary independent variable. Concretely, we compare graph families ranging from a single-agent baseline to waterfall-style pipelines and iterative verification loops, while holding constant the underlying model, benchmark, and role set as much as possible. This experimental design permits a more controlled account of when additional coordination improves performance and when it merely purchases more opportunities for verification through higher token expenditure.

To answer that question, we complement benchmark evaluation with a game-theoretic analysis of contribution and incentives. We model successful task completion as a team payoff. Prior work in multi-agent reinforcement learning has shown that Monte Carlo Shapley estimation can provide useful global contribution signals even when exact computation is expensive, while also emphasizing that such values explain average contribution rather than any single trajectory (Heuillet et al., 2022). We analyze whether a graph supports stable high-effort verification under token costs by using a utility-based deviation test grounded in standard multi-agent reasoning about incentives and equilibrium (Shoham and Leyton-Brown, 2008). It is also increasingly well aligned with recent work arguing that game-theoretic concepts such as cooperation, credit allocation, and equilibrium analysis are becoming useful tools for understanding LLM systems and LLM-based agents more broadly (Sun et al., 2025).

Our central claim is that interaction topology should be studied not only as an architectural choice, but also as an incentive mechanism. A graph determines where verification occurs, who bears its token cost, and how strongly an agent can rely on others to catch errors later. By jointly measuring pass@1, token usage, and contribution/incentive patterns, this paper aims to move the study of multi-agent code generation from pipeline-level comparison toward a more controlled and explanatory analysis of why some structures work better than others. We define utility as the balance between task success and cost of tokens of effort, and we use it to determine whether a graph structure supports an approximate Nash Equilibrium where all agents are incentivized to perform high effort verification under constrained compute costs (Shoham & Leyton-Brown, 2008).

Our main contributions are:

(1) We introduce a controlled experimental framework for analyzing multi-agent code generation in which interaction topology is isolated as the principal design variable, while models, MBPP tasks, and role prompts are kept fixed across graph families.
(2) We cast multi-agent code generation in a game-theoretic framework by defining agent utility as a shared task-success reward minus individual token expenditure, and we operationalize incentive stability through empirical ε-Nash-style deviation tests together with Shapley-value-based contribution analysis.
(3) We provide a systematic comparison of multiple topology families implemented in a common LangGraph framework, showing that cyclic feedback structures achieve a stronger accuracy-cost tradeoff than more elaborate parallel designs, thereby offering evidence that architectural complexity is useful only when it improves corrective interaction rather than merely increasing communication overhead.

2 Related Work

Recent work has explored increasingly structured forms of collaboration. General multi-agent frameworks such as MetaGPT, AutoGen, CAMEL, and ChatDev formalize role specialization and inter-agent communication, while code-focused systems such as AgentCoder, Reflexion, MapCoder, CodeCoR, and AdaCoder show that iterative cycles of

generation, critique, execution, and revision can materially improve correctness (Hong et al., 2023; Wu et al., 2024; Li et al., 2023a; Qian et al., 2023; Huang et al., 2023; Shinn et al., 2023; Islam et al., 2024; Pan et al., 2025; Zhu et al., 2025). Collectively, this literature suggests that coordination structure matters, but much of it still evaluates full pipelines in which prompts, roles, tools, memory, and stopping rules change together.

A particularly relevant recent development is the move from fixed topology design to adaptive topology selection. GPTSwarm treats a multi-agent system as an optimizable graph whose connectivity can be improved through learning (Zhuge et al., 2024). More recently, AMAS argues that even an optimized static graph may not be uniformly best across inputs: different samples may favor different communication structures, and a lightweight selector can choose among candidate graphs at inference time (Leong et al., 2025). Across question answering, mathematical reasoning, and code-generation benchmarks, including HumanEval, AMAS reports gains over both single-agent baselines and prior multi-agent systems while maintaining latency comparable to GPTSwarm (Leong et al., 2025). This is an important result because it strengthens the case that topology is not merely an implementation detail, but a substantive factor in system performance.

At the same time, adaptive-topology work does not eliminate the need for controlled structural analysis. Systems such as GPTSwarm and AMAS remain end-to-end proposals in which graph optimization, graph selection, prompting choices, training procedures, candidate generation, and evaluation pipelines are introduced together. As a result, they provide strong evidence that communication structure matters, but they do not fully isolate which gains are attributable to topology itself rather than to accompanying changes in prompts, feedback channels, or optimization procedures. The same concern applies more broadly across the multi-agent code-generation literature. A waterfall pipeline, an iterative test-repair loop, and a richer cyclic workflow differ not only in graph form, but also in when errors are exposed, where information is transformed, who performs verification, and how much downstream recovery is possible after an early mistake. Surveys of LLM-based software engineering agents and LLM multi-agent systems reinforce this point by treating communication structure, coordination, planning, and memory as central and interacting dimensions of design (Liu et al., 2024; Wang et al., 2025).

3 Methodology

3.1 Problem Formulation

We model multi-agent code generation as a cooperative game in which a set of specialized agents collaborates to solve a shared programming task. The agent set is:

$$\mathcal{N} = \{PL, C, R, T\}, \tag{1}$$

corresponding to the Planner, Coder, Reviewer, and Tester. For a task x and graph topology g, the system executes a structured interaction protocol for at most K rounds and returns an outcome:

$$Y \in \{\text{pass}, \text{fail}\}, \tag{2}$$

where success is determined by whether the final program passes all unit tests.

Each agent $i \in \mathcal{N}$ selects an effort level:

$$e_i \in \{0, 1\}, \tag{3}$$

where $e_i = 1$ denotes high effort and $e_i = 0$ denotes low effort. Higher effort corresponds to more rigorous reasoning, verification, and revision behavior, but also incurs greater computational cost. Let T_i denote the token expenditure of

agent i. We define the utility of agent i as:

$$U_i = V \cdot \mathbf{1}[Y = \text{pass}] - \lambda T_i, \tag{4}$$

where $V > 0$ is the shared value of task success, $\mathbf{1}[Y = \text{pass}]$ is an indicator equal to 1 when the generated solution passes all tests and 0 otherwise, and $\lambda > 0$ is a cost scalar that controls how heavily computation is penalized.

This utility function captures the central tradeoff in our study. On the one hand, all agents benefit from successful completion because the reward term is shared across the team. On the other hand, each agent individually pays for its own computational effort through token usage. A rational agent therefore faces a tension between contributing more effort to increase the probability of success and reducing its own token cost by acting more cheaply.

Our goal is to analyze whether a graph topology can sustain high-effort verification as a stable behavior profile. Let:

$$e^* = (1, 1, 1, 1) \tag{5}$$

denote the all-high-effort profile. We say that a topology is approximately stable if, for every agent i, unilateral deviation to low effort does not improve utility by more than a small tolerance ε. Formally, we test whether:

$$U_i(1, e^*_{-i}) \geq U_i(0, e^*_{-i}) - \varepsilon. \tag{6}$$

Equivalently, no agent should have a strong incentive to slack off while relying on the rest of the graph to preserve overall success. This allows us to interpret topology not only as an architectural design choice, but also as an incentive mechanism that shapes whether verification effort is sustained or free-ridden.

3.2 Operationalizing Agent Roles and Effort

We instantiate four standard agent roles. The Planner decomposes the task logic and proposes a high-level solution strategy. The Coder translates the plan into executable Python code. The Reviewer performs static analysis, checks for edge cases, and identifies logical or stylistic weaknesses. The Tester executes unit tests and surfaces trace information that can guide repair.

Since we use API-based LLM systems, we could not fine-tune model weights, thus effort cannot be manipulated directly at the parameter level. Instead, we operationalize effort through generation constraints to approximate strategic deviation. High effort ($e_i = 1$) gives the agent explicit reasoning-oriented prompts, a more permissive generation setting, and a larger token budget. In our implementation, high-effort agents receive a budget of 2048 tokens and are prompted to reason step by step. Low effort ($e_i = 0$) is implemented by sharply restricting the agent's capacity: the prompt is minimal, decoding is greedy with temperature 0.0, and the token budget is reduced to 516. This intervention simulates a low-investment agent that produces concise, weakly verified outputs and effectively "throws code over the wall" to downstream agents.

3.3 Graph configurations

Our central methodological choice is to isolate topology as the primary independent variable. To do so, we hold constant the base model, the task distribution, and the role prompts, and vary only the communication graph that governs how information flows among agents.

We compare eight representative topologies implemented in a shared LangGraph framework. These graph families span increasingly rich coordination structures:

- **G0: Baseline (0-shot).** A single universal agent maps the input directly to an output without explicit reflection or coordination.
- **G1: Waterfall.** A strictly linear pipeline PL → C → R → T with no backtracking.
- **G2: AgentCoder Loop.** A tight coder–tester repair loop in which execution feedback determines whether the code is revised again.
- **G2.5: Reviewer Repair.** Test failures are routed through a Reviewer for static diagnosis before returning to the Coder for patching.
- **G3: MapCoder Cycle.** Failures can trigger a larger replanning cycle that routes information back to the Planner before another coding attempt.
- **G4: Parallel Judge.** Multiple Coders generate solutions in parallel, after which a Judge selects the candidate sent to testing.
- **G5: Adversarial Debate.** A Red Team critiques the Coder's solution and a Blue Team responds or patches before testing.
- **G6: Hierarchical Setup.** A Manager delegates subtasks to Workers, whose outputs are combined by an Aggregator before testing.

3.4 Experimental Protocol and Evaluation

All graph families are implemented in a shared LangGraph framework so that the primary difference across conditions is the interaction topology rather than the orchestration environment. Consistent with our controlled-design objective, we keep the backbone model, task set, and role prompts fixed within each comparison and vary only the communication graph governing information flow among agents. We evaluate the system on 50 targeted programming problems drawn from the MBPP benchmark and use telemetry callback handlers to trace prompt and completion tokens throughout execution, allowing precise measurement of computation cost.

To examine whether topology effects are robust across model capability, we run the graph families on two backbone models, GPT-5.4 Nano and Qwen2.5. No model parameters are fine-tuned. Instead, all experiments use the same role definitions and generation framework, so that differences in performance can be attributed as directly as possible to topology rather than to prompt engineering or model adaptation.

We report two primary system-level metrics: pass@1, which measures whether the final generated solution passes the unit tests, and total token cost, which measures the aggregate compute consumed by the graph. These two metrics capture the core accuracy–efficiency tradeoff. In addition, because our framing is game-theoretic rather than purely benchmark-driven, for each graph, we perform an empirical best-response test by forcing one role to low effort while others remain high and estimating the utility change ΔU_i. A graph supports stable high-effort verification if no role benefits from unilateral deviation.

We perform an empirical Nash-style deviation test. For each topology, we begin from the high-effort profile $e^* = (1, 1, 1, 1)$, then force one agent at a time to low effort while holding the others fixed. We recompute utility under the deviated profile and check whether the deviating agent benefits from the reduction in effort cost without substantially harming the team's probability of success. If unilateral deviation improves utility, the topology is considered incentive-unstable because it permits free-riding; otherwise, it supports approximately stable high-effort verification.

4 Discussion

The results show that interaction topology has a substantial effect on the accuracy–compute tradeoff in multi-agent code generation. Across both backbone models, the strongest structures are not the most elaborate ones, but the ones that create short, actionable repair loops between generation and execution. On GPT-5.4 Nano, the best-performing topology is G3 (MapCoder), which reaches 87.8% pass@1 at an average cost of 1,658 tokens. It is closely followed by G2 (AgentCoder) at 84.0% with only 987 tokens, and G2.5 (Reviewer Repair) at 83.7% with 1,220 tokens. These loop-based designs clearly dominate the simpler G1 Waterfall, which reaches only 74.0% at 1,204 tokens, and they also dominate the more elaborate parallel and hierarchical graphs. In particular, G4 (Parallel Judge) achieves only 72.0% despite requiring 4,717 tokens, G5 (Adversarial Debate) reaches 78.0% at 5,506 tokens, and G6 (Hierarchical Setup) remains at 74.0% even after consuming 7,008 tokens.

A similar pattern appears on Qwen2.5 Coder 7B, although the absolute performance is lower and the best frontier shifts slightly. The strongest topology on this model is G2.5, which reaches 56.0% pass@1 at 1,511 tokens. G2 and G3 both reach 42.0%, but G2 does so at 1,133 tokens, whereas G3 requires 2,459 tokens. The non-cyclic alternatives remain clearly weaker: G1 reaches 26.0% at 958 tokens, G4 only 4.0% at 547 tokens, G5 reaches 30.0% at 686 tokens, and G6 reaches 30.0% at 270 tokens. This is an important qualification.

A key implication is that additional communication is only useful when it improves corrective interaction. In the cyclic graphs, execution feedback is routed back to the agent or subroutine that can directly act on it, so token expenditure is concentrated on repairing concrete failures rather than on maintaining coordination. This explains why G2, G2.5, and G3 remain near the efficient frontier in the GPT-5.4 Nano results, achieving substantially higher pass@1 than the baseline and waterfall structures without the large token overhead incurred by G4, G5, and G6. The same principle appears in the Qwen2.5 results, although the weaker backbone changes which cyclic design is most effective. On the stronger model, the larger replanning cycle in G3 yields the highest accuracy, suggesting that repeated global revision can be productively exploited when the underlying model can sustain it. On the weaker model, however, G2.5 performs best, indicating that a lighter-weight reviewer-mediated repair loop may be more beneficial than a full replanning cycle when model capability is limited. Taken together, these results suggest that graph complexity interacts with base-model strength: richer cycles can extract more value from stronger models, while weaker models benefit more from targeted local correction than from deeper coordination.

The incentive analysis adds an important layer beyond the standard accuracy–cost comparison. The Nash-style stability scores are high for the non-redundant and loop-based structures: G1 = 84.0%, G2 = 78.0%, G2.5 = 81.6%, and G3 = 77.6%. By contrast, the redundant structures are far less stable: G5 = 26.0% and G6 = 10.0%. This gap is large enough that it should be treated as a structural property of the topology rather than as noise. In G2, G2.5, and G3, low-effort deviations are quickly exposed through failed execution or unsuccessful repair, which sharply reduces the shared payoff. As a result, agents cannot easily save tokens while still benefiting from successful task completion. By contrast, in redundant parallel or hierarchical structures such as G5 and G6, one weak branch can often be masked by the output of another branch or filtered out by an aggregation step. This creates room for free-riding: an agent may reduce its own effort cost while still enjoying the group reward if the overall system succeeds. The sharp drop in Nash-style stability for G5 and G6 is therefore not incidental; it reflects a structural decoupling between local effort and final payoff.

An interesting nuance is that stability alone is not sufficient for strong overall performance. The waterfall topology G1 is relatively stable, because responsibility is not redundant and upstream errors remain consequential. However, it

still underperforms the cyclic graphs on accuracy because it lacks an effective mechanism for iterative correction once an error has propagated downstream. This distinction is important. A useful multi-agent topology must do two things simultaneously: it must preserve individual accountability, and it must provide a pathway for test signals or review signals to trigger repair. G1 largely satisfies the first condition but not the second, whereas G2, G2.5, and G3 satisfy both. This helps explain why the loop-based designs dominate the broader tradeoff space.

5 Conclusion

Our results show that topology has a first-order effect on the accuracy–compute tradeoff. Across both GPT-5.4 Nano and Qwen2.5 Coder 7B, the most effective structures are not the most elaborate parallel or hierarchical graphs, but the cyclic repair topologies that tightly connect generation with execution feedback. In particular, G2, G2.5, and G3 consistently occupy the most favorable region of the tradeoff frontier, achieving substantially stronger pass@1 performance than baselines and linear pipelines while avoiding the large coordination costs of more redundant graph designs. At the same time, the best-performing cyclic structure depends on model capability: stronger models benefit more from richer replanning cycles, whereas weaker models benefit more from lighter-weight local correction. This suggests that graph complexity is valuable only when the underlying model can use the additional coordination productively.

Beyond standard benchmark evaluation, our game-theoretic analysis shows that topology also shapes incentive alignment. Loop-based structures remain comparatively stable because low-effort deviations are exposed quickly through failed execution or unsuccessful repair, making free-riding difficult. In contrast, redundant parallel and hierarchical topologies create opportunities for agents to reduce effort while still benefiting from group success, which weakens verification incentives and lowers stability. These results support the broader claim that interaction topology should be understood not only as an architectural design choice, but also as an incentive mechanism that determines where errors are exposed, who is responsible for correction, and whether local effort remains essential to overall success.

Taken together, our findings move the study of multi-agent code generation from pipeline-level comparison toward a more controlled and explanatory account of why some coordination structures work better than others. The main lesson is simple: architectural complexity is useful only when it strengthens corrective interaction without weakening accountability. For the design of future multi-agent coding systems, this suggests that the central question is not how many agents a graph contains, but whether its structure turns feedback into efficient, responsibility-preserving repair.

6 Limitations and Future Work

6.1 Scope

Our study is intentionally narrow in order to obtain a controlled comparison of topology. We fix the backbone model, task setting, and role prompts, and evaluate a set of representative graph families on 50 targeted MBPP problems because of the computational constraints. This design is a methodological strength because it isolates interaction topology as the principal independent variable, but it also limits the scope of the conclusions. In particular, our results should be interpreted as evidence about topology under a constrained code-generation setting, rather than as a general statement about all multi-agent software engineering workflows. Realistic deployment scenarios often involve longer horizons, external tools, retrieval, multi-file repositories, and richer forms of coordination than are present in MBPP-style task solving.

A second scope limitation is that our evaluation focuses on a static set of hand-designed graph families. This is appropriate for the paper's goal of controlled structural comparison, but it means we do not test whether the best

topology varies systematically across inputs. Recent work on adaptive topology selection suggests that no single graph structure is uniformly optimal across all instances; rather, different inputs may favor different communication patterns, and a selector can improve performance by choosing among candidate graphs at inference time (Leong et al., 2025). Our study therefore clarifies the behavior of several important topology classes, but it does not yet address instance-level topology selection or adaptation.

6.2 Approximation of the incentive formulation

Our game-theoretic formulation is deliberately simplified. We model multi-agent code generation as a cooperative game with shared task reward and individual token cost, and we use this abstraction to ask whether a topology supports approximately stable high-effort verification. This captures the main strategic tension in our setting, but it does not constitute a full equilibrium theory of LLM agents. In particular, our "effort" variable is not learned or chosen endogenously by a rational policy. Instead, it is operationalized through inference-time constraints such as prompt style, decoding settings, and token budgets. As a result, the deviation analysis should be interpreted as an empirical intervention-based approximation of incentive stability, not as a formal proof that real LLM agents optimize utility in the game-theoretic sense.

6.3 Future directions

One natural next step is to extend the evaluation beyond MBPP and beyond a fixed set of targeted tasks. Future work should test whether the same structural patterns hold on harder benchmarks and more realistic software engineering problems, such as repository-level repair, tool-augmented programming, and longer-horizon debugging workflows. Such evaluations would clarify whether the superiority of cyclic repair structures persists when coordination must unfold over larger contexts and more complex execution traces. This would also align the study more closely with recent multi-agent LLM work that evaluates across heterogeneous task families rather than within a single tightly controlled benchmark (Leong et al., 2025).

A second direction is to move from static comparison to adaptive topology control. Our results show that loop-based structures dominate in general, but they also suggest that the best cyclic variant depends on model capability: richer replanning appears more useful for stronger models, while lighter reviewer-mediated repair may be more effective for weaker ones. This naturally raises the question of whether topology should be chosen dynamically based on the input instance, model capability, or intermediate execution signals. AMAS provides an important precedent by showing that per-sample graph selection can outperform static graphs across diverse workloads, suggesting a promising path for combining our controlled analysis with adaptive deployment strategies (Leong et al., 2025).

A third direction is to refine the incentive analysis itself. Our current framework studies unilateral deviation under a fixed high-effort reference profile, but richer strategic phenomena may depend on sequential adaptation, nonstationarity, and the order in which agents or subroutines update their behavior. In multi-agent reinforcement learning, these issues are significant enough that dedicated methods have been proposed to model agent-by-agent updates and update-order effects directly (Wang et al., 2023). Bringing similar ideas into LLM-based agent systems could lead to more realistic models of incentive stability, especially in graphs where behavior unfolds over multiple correction stages rather than a single deviation test.

Finally, future work could relax the assumption that coordination is entirely specified by hand-crafted roles and edges. Classic work on multi-agent communication emphasizes that effective protocols may need to be discovered

rather than fully pre-imposed, especially when agents operate under partial observability and communication bottlenecks (Foerster et al., 2016). In the context of LLM-based code generation, this suggests a broader research agenda in which not only topology, but also the semantics of inter-agent messages, the allocation of verification responsibility, and the granularity of feedback loops are learned or adapted jointly. Our paper takes a first step by isolating topology; a fuller account of multi-agent code generation will likely require studying how topology, communication protocol, and incentive structure co-evolve.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021).

[2] Jakob N. Foerster, Yannis M. Assael, Nando de Freitas, and Shimon Whiteson. 2016. Learning to Communicate with Deep Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:1605.06676* (2016).

[3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven K. S. Yau, Zijian Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *arXiv preprint arXiv:2308.00352* (2023).

[4] Dong Huang, Jie M. Zhang, Michael Luck, Qing Bu, Yue Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. *arXiv preprint arXiv:2312.13010* (2023).

[5] M. A. Islam, M. E. Ali, and M. R. Parvez. 2024. MapCoder: Multi-agent code generation for competitive problem solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4912–4944.

[6] H. Y. Leong, Y. Li, Y. Wu, W. Ouyang, W. Zhu, J. Gao, and W. Han. 2025. AMAS: Adaptively determining communication topology for LLM-based multi-agent system. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. 2061–2070.

[7] Guohao Li, Hasan Hammoud, Hani Itani, Dmitry Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36.

[8] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen, L. Zhang, and Y. Lou. 2024. Large language model-based agents for software engineering: A survey. *arXiv preprint arXiv:2409.02977* (2024).

[9] R. Pan, H. Zhang, and C. Liu. 2025. CodeCoR: An LLM-based self-reflective multi-agent framework for code generation. *arXiv preprint arXiv:2501.07811* (2025).

[10] Chen Qian, Wei Liu, Hongyi Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Junda Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ChatDev: Communicative Agents for Software Development. *arXiv preprint arXiv:2307.07924* (2023).

[11] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36.

[12] X. Wang, Z. Tian, Z. Wan, Y. Wen, J. Wang, and W. Zhang. 2023. Order matters: Agent-by-agent policy optimization. *arXiv preprint arXiv:2302.06205* (2023).

[13] Y. Wang, Y. Pan, Z. Su, Y. Deng, Q. Zhao, L. Du, T. H. Luan, J. Kang, and D. Niyato. 2024. Large model based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends. *arXiv preprint arXiv:2409.14457* (2024).

[14] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. 2024. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the First Conference on Language Modeling*.

[15] Y. Zhu, C. Liu, X. He, X. Ren, Z. Liu, R. Pan, and H. Zhang. 2025. AdaCoder: An adaptive planning and multi-agent framework for function-level code generation. *arXiv preprint arXiv:2504.04220* (2025).

[16] Mingchen Zhuge, Weizhao Wang, Louis Kirsch, Francesco Faccio, Dmitry Khizbullin, and Jürgen Schmidhuber. 2024. GPTSwarm: Language Agents as Optimizable Graphs. In *Proceedings of the 41st International Conference on Machine Learning*, Vol. 235. 62743–62767.